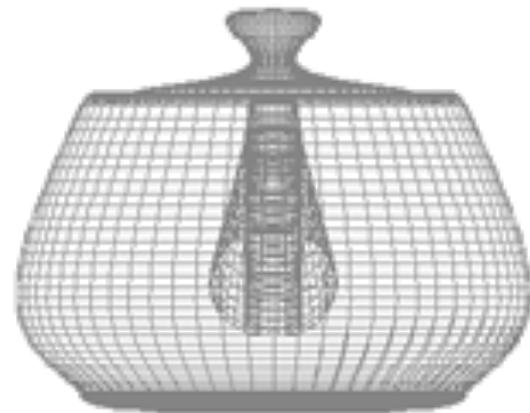
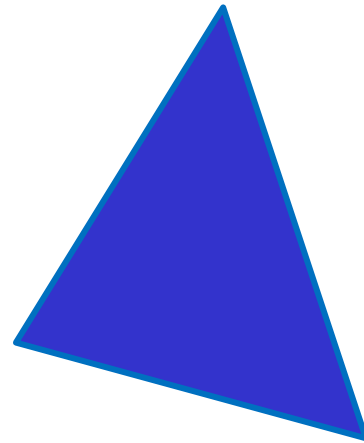
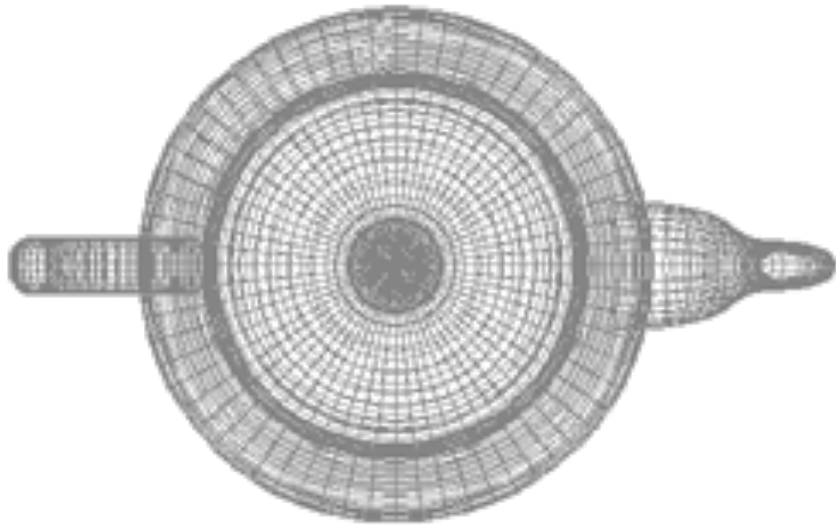


Laboratorio 4

OpenGL Hello Triangle



Capitolo 1 - Disegnare un triangolo

template di lavoro: `00_hello-gl.cc`

soluzione: `01_hello-triangle.cc`

Piano di lavoro

- Costruiamo un programma che disegna un triangolo usando OpenGL:
 1. Creiamo una canvas e la mettiamo a disposizione di OpenGL
 2. Creiamo la scena e la trasmettiamo alla GPU
 3. Spieghiamo a OpenGL che regola usare per fare il rendering (shaders)
 4. Disegniamo

Le librerie grafiche sono un delirio!

- Quale libreria?
 - Metal, Vulkan, OpenGL (innumerevoli tipi e versioni), Direct3D (11 o 12)...
 - Non è banale capire quale usare e perché.
 - Nessuna di queste è facile o intuitiva da usare.
- Scegliamo OpenGL 4.1!
 - Nonostante lo sviluppo di OpenGL sia oramai fermo, continua ad essere ampiamente supportato per lo sviluppo di applicazioni grafiche cross-platform, anche su hardware recente (ultima versione di OpenGL: 4.6 del 2017, ma la 4.1, del 2010, è l'ultima supportata su MacOS).
 - Complicato fare il set-up e farraginoso da usare. Le alternative restano comunque peggiori!
- E per il futuro?
 - Andrebbe considerato WebGPU, un abstraction layer costruito sopra Vulkan, Metal e Direct3D12, pensato per i browser, ma con librerie native Rust e C++ per il desktop

Le librerie grafiche sono un delirio!

- Come mai è tutto così complicato?
 - Fare grafica 3D richiede di fare tanti, tanti calcoli. Per farli, bisogna usare hardware specializzato, molto complesso, con tante unità di computazione che lavorano in parallelo.
 - Sperare che il proprio software di grafica accelerata giri “correttamente” sul computer del primo che passa vuol dire dover prevedere un’enorme varietà:
 - Di schede video.
 - Ognuna con diverse versioni e tipologie di driver.
 - Sotto diversi sistemi operativi.
 - In OpenGL, questo vuol dire che il proprio programma deve chiedere al sistema cosa sia disponibile e caricare dinamicamente, a run time, i pezzi che gli servono. Per fortuna ci sono delle librerie che se ne occupano per noi. Noi useremo GLAD insieme a SFML.

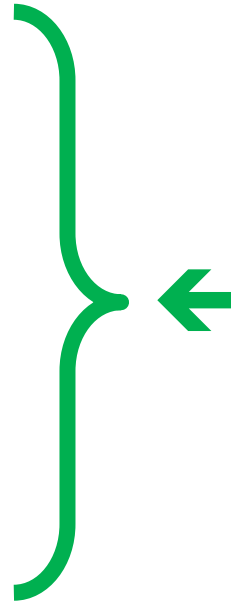
Open GL: Macchina a stati

- OpenGL si basa su un proprio stato interno. Qualche esempio:
 - `clear color` attivo al momento.
 - `depth test` (abilitato o no, quale funzione usare).
 - `blending` (abilitato o no, quale funzione usare).
 - Quale buffer o spazio di memoria è al momento attivo per trasferire o modificare informazioni (aspetto importantissimo, in quanto ha a che fare con la comunicazione fra memoria di sistema e memoria video).
- Molti comandi non hanno effetto immediato, ma preparano lo stato influenzando quelli che poi saranno comandi successivi!
 - Lo stato interno contiene a tutti gli effetti dei parametri impliciti delle funzioni di libreria, questo rende il codice complesso da creare e seguire.
 - Esempio di query dello stato:

```
double v[4];  
glGetDoublev (GL_COLOR_CLEAR_VALUE, v);
```

Passi fondamentali

1. Inizializzare OpenGL
2. Definire **cosa** disegnare
3. Definire **come** disegnare
4. Main loop con eventi base
5. ... Disegnare



C++
OpenGL+GLSL
GLAD
SFML

Ingredienti principali

- **Geometria minimale**: un triangolo, di un solo colore.
- **Vertex shader** minimale: passa il vertice tale e quale.
- **Fragment shader** minimale: tutti i frammenti di quel colore.

Primo passo: inizializzare

- Potenzialmente, un incubo. Bisogna:
 - Includere OpenGL.
 - Farsi dare una finestra dal sistema, gestirne gli eventi (SFML).
 - Caricare dinamicamente i pezzi di OpenGL necessari (GLAD).
 - Inclusioni, inizializzazioni e caricamenti devono essere fatti in ordine corretto, altrimenti le cose smettono di funzionare!
 - Bisogna controllare cosa è stato effettivamente trovato e caricato sul sistema su cui sta girando il software, non c'è alcuna garanzia che le richieste siano esaudite.
 - In software più sofisticati, in questa fase si decide dinamicamente se abilitare o no certe feature in base a cosa il sistema supporta.
- **Compile il template di partenza `00_hello-gl.cc`, controllate se gira e cosa trova!!!**

Cenni su GLAD

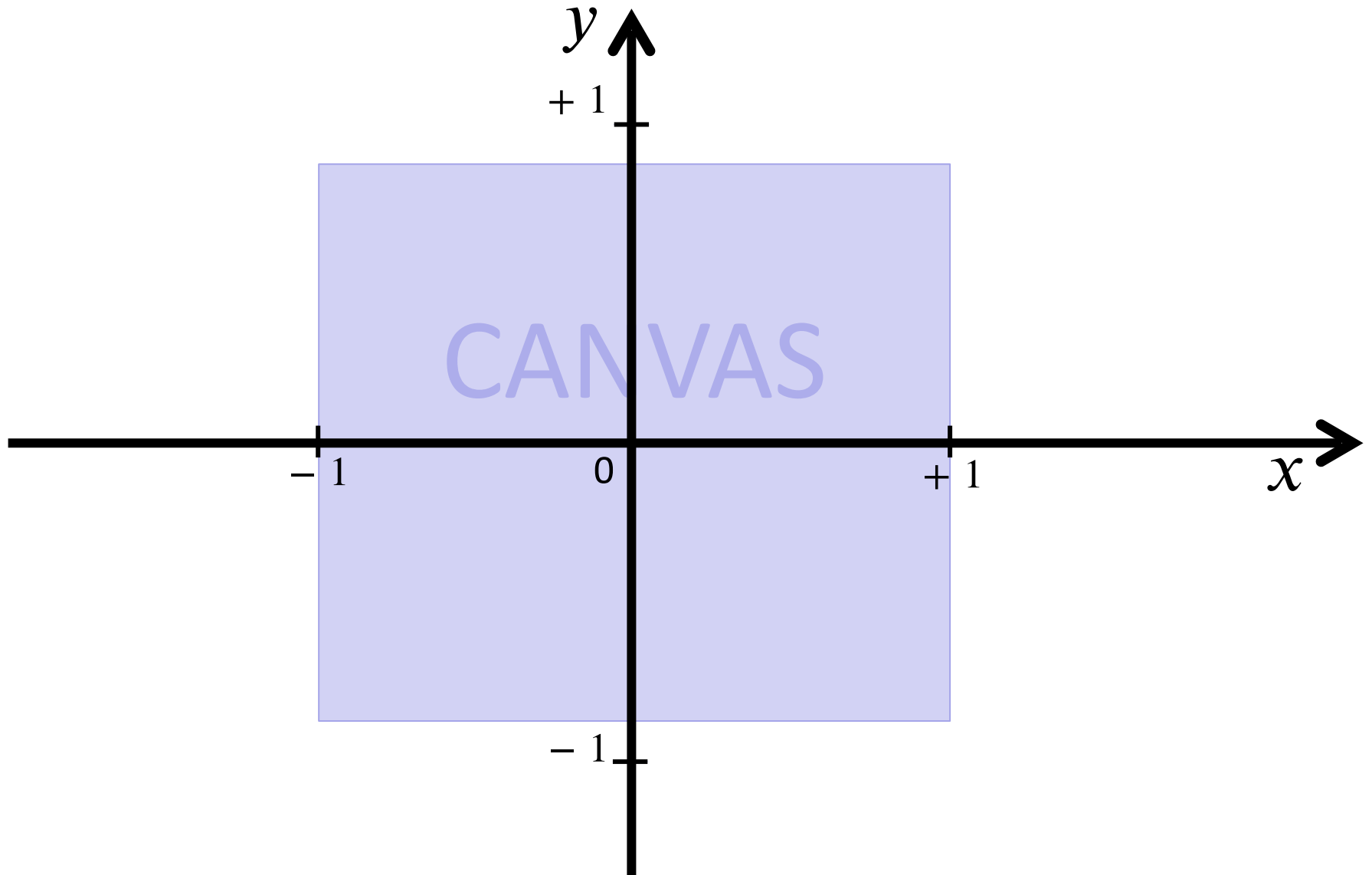
- Come funziona il sistema GLAD?
 - È un software open source, con un'interfaccia web, tramite la quale si chiede quale versione di OpenGL si vuole avere e con quali estensioni. L'interfaccia genera una libreria (opzionalmente header only) da includere, che cerca di effettuare il caricamento dinamico di quanto richiesto.
 - Se in futuro aveste bisogno di una versione di OpenGL diversa, o estensioni specifiche, bisogna generare un nuovo header glad.
 - Al momento non c'è bisogno di fare nulla, forniamo noi un header glad già costruito, che carica una configurazione core 4.1, senza estensioni.
- In pratica: usate il template fornito; dovrebbe risolvere tutti questi problemi con un minimo comune denominatore accettabile.

Cosa disegniamo?

- Un solo triangolo in 2D (ossia: nel piano $z = 0$).
- Direttamente in **clip coordinates** (nessuna trasformazione).
- Non indicizzato (**soup**), ovvero tre vertici consecutivi fanno un triangolo.
- In C++:

```
float points[] = {  
    0.0f,  0.5f,  0.0f,  // x,y,z of first point.  
    0.5f, -0.5f,  0.0f,  // x,y,z of second point.  
    -0.5f, -0.5f,  0.0f  // x,y,z of third point.  
};
```

Clip Space



OpenGL: Object Names

- Abbiamo bisogno di oggetti nella RAM video:
 - Di solito, duplicati da oggetti nella RAM di sistema.
 - Per esempio, **vertex buffer**, **element buffer**, **programs**, **textures**, etc...
- Come li riferiamo, manipoliamo, e trasferiamo?
 - Letture, scritture devono passare attraverso il BUS di sistema!
- Si usano degli «**object names**»:
 - Un indice di una certa istanza di un dato oggetto, come un array buffer, etc.)
 - In pratica, un numero intero!
 - Il nome è in System RAM, quindi, semplicemente un intero in una variabile di programma, che punta ad un oggetto in Video RAM.
 - Il tipo di un oggetto è descritto da costanti dello stato di OpenGL, come **GL_CONSTANT_<SOMETHING>**.
 - I nomi degli oggetti sono generati da funzioni come «**create <something>**» o «**gen <something>**».
 - I nomi sono distrutti e liberati tramite funzioni come «**delete <something>**».

OpenGL: Binding

- Situazione:
 - Molti comandi agiscono su un oggetto implicito nello stato di OpenGL.
 - Per esempio, su un **array buffer**.
 - Ci sono diverse istanze di quel tipo di oggetto.
 - Per esempio, diversi **array buffer** in memoria video.
- Come si sceglie su quale istanza agisce un comando?
 - In un dato momento, solo un'istanza è «**bound**» nello stato OpenGL!
 - Ad esempio, un **array buffer** è **bound**, un **program** è **bound**, etc.
 - Esiste un comando “**bind**” per attivare una specifica istanza nello stato.
 - Tutti i comandi successivi agiscono su quell'istanza, fino a quando non viene cambiata.

OpenGL: Caricamento dei dati in video RAM

- Generazione di **array buffer** e copia dei dati:

Vogliamo solo un buffer.

Qui sarà restituito il **nome** del buffer in Video RAM, da usare per riferirlo in seguito.

GenBuffers: creazione!

```
glGenBuffers (1, &vbo);  
glBindBuffer (GL_ARRAY_BUFFER, vbo);  
glBufferData (GL_ARRAY_BUFFER,  
              sizeof(vertices),  
              vertices,  
              GL_STATIC_DRAW);
```

lunghezza e puntatore ai dati in system RAM da copiare

BufferData: invio dei dati alla GPU

BindBuffer: tutti i comandi successivi che usano un GL_ARRAY_BUFFER implicito, useranno questo.

(suggerimento al driver video: pensiamo di voler cambiare questi dati solo raramente)

OpenGL: Organizzazione dei dati in video RAM

- **Vertex Array Object:**
 - Il **VAO** è un contenitore che descrive dati che sono in memoria video e come interpretarli; per esempio: dove si trovano le coordinate e i colori di ogni singolo vertice.
 - Non deve obbligatoriamente descrivere tutti i dati, anzi. È pensato per poter attivare delle viste sui dati che possono essere parziali. Ad esempio, descrivere solo i pezzi che ci servono in quel momento.
 - Possono essercene più di uno: diversi **VAO** possono offrire viste differenti sull'insieme dei dati che sono stati trasferiti alla memoria video.
- In questo caso, aggiungiamo al **VAO** un solo attributo:
 - Le coordinate dei vertici, contenute nel **VBO**.
- Le informazioni nel **VAO** servono al **Vertex Puller** per capire:
 - Quali tra i dati caricati in memoria RAM devono essere passati alla pipeline di rendering
 - Come sono organizzati gli attributi relativi a ogni vertice
- Il **VAO** deve essere coerente con gli **shader** (vedi slide successive)

OpenGL: Organizzazione dei dati in video RAM

Chiedo a OpenGL di
generare un **VAO**



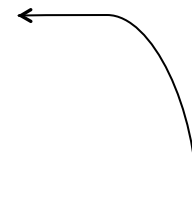
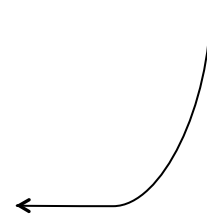
```
// Generate a vertex array object (VAO) named vao:
GLuint vao;
glGenVertexArrays (1, &vao);

// Bind vao as the current VAO in the OpenGL state:
glBindVertexArray (vao);

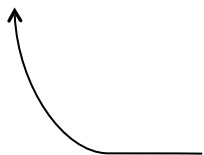
// Attach data inside current VBO to current VAO, as
// attribute 0, where each one is made up of 3 floats,
// not normalized, tightly packed:
glVertexAttribPointer (0, 3, GL_FLOAT, GL_FALSE, 0, NULL);

// Enable attribute 0 of current VAO for the vertex shader:
glEnableVertexAttribArray (0);
```

Rendo questo **VAO**
attivo



Rendo l'**attributo 0**
disponibile: il vertex
puller dovrà trasferire
questi dati alla pipeline

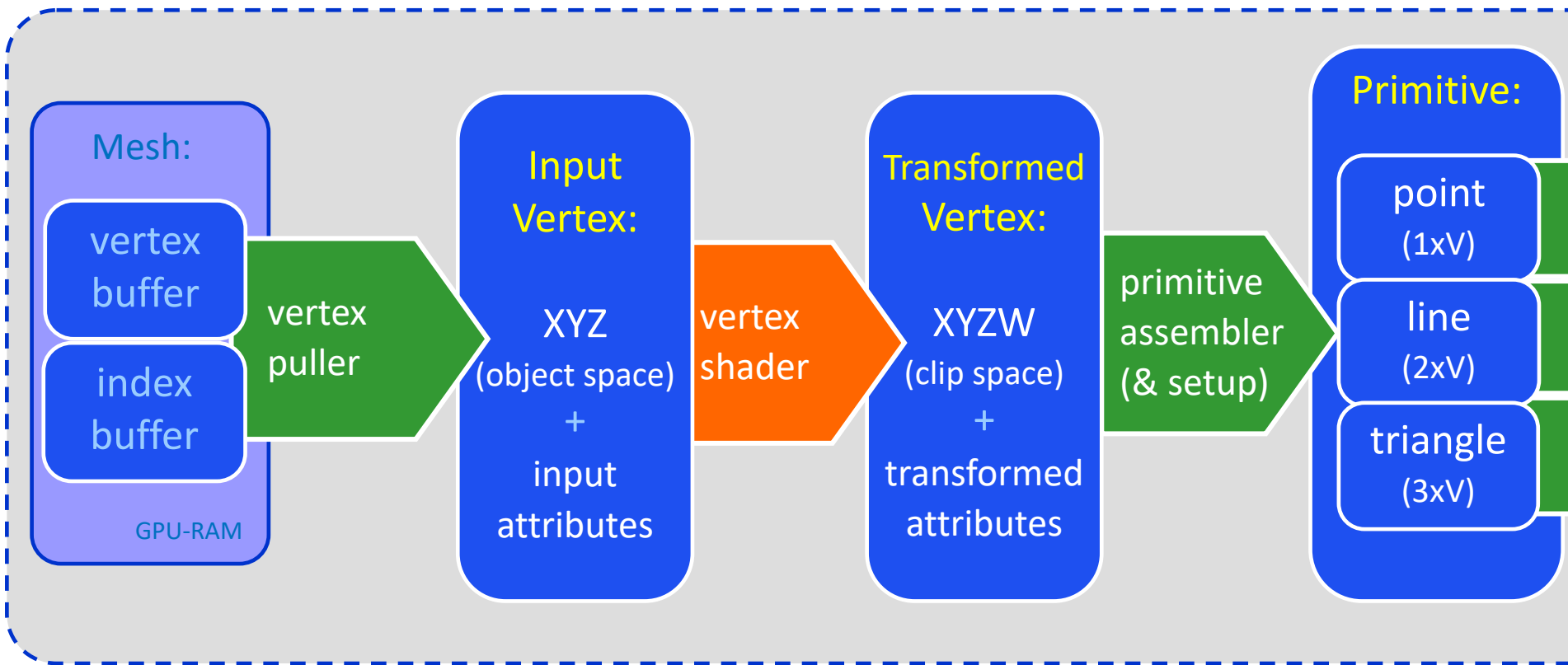


Specifico la struttura
dell'**attributo 0** nel **VBO**
corrente: questa viene
memorizzata nel **VAO**

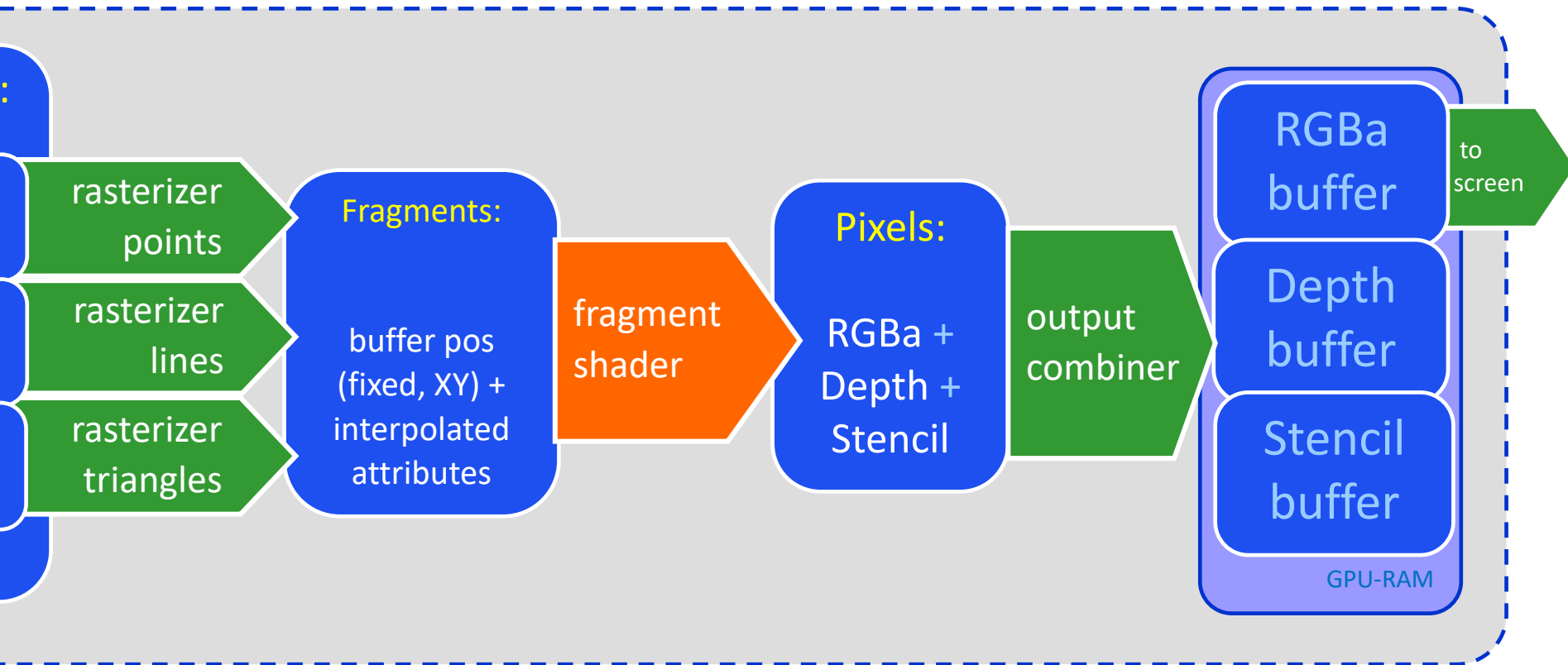
Come disegniamo?

- Definiamo gli shader!
 - **vertex shader** (viene eseguita un'istanza per ogni vertice).
 - **fragment shader** (viene eseguita un'istanza per ogni frammento).
- Linguaggio specifico: **GLSL**
 - «C-like»: niente oggetti.
 - Dinamicamente compilato in linguaggio macchina della GPU.
 - Ogni shader gira sulla GPU, al suo punto designato nella pipeline.
 - Tipi nativi vec2, vec3, vec4, mat4 e algebre corrispondenti.
- Nel nostro caso, shader minimali:
 - **vertex shader**: Passiamo le coordinate come le riceviamo: nessuna trasformazione, o meglio, la trasformazione è l'identità.
 - **fragment shader**: Passiamo un colore costante: tutto viene disegnato di quel colore.

Reminder: la pipeline grafica



Reminder: la pipeline grafica



Shader minimali

vertex shader:

L'attributo 0 del VAO viene collegato a vp in questo shader

```
layout(location = 0) in vec3 vp;  
void main (void) {  
    gl_Position = vec4 (vp, 1.0);  
}
```

nome predefinito GLSL

XYZ

W (affine)

GLSL

fragment shader:

```
out vec4 FragColor;  
void main (void) {  
    FragColor = vec4 (0.0, 0.0, 1.0, 1.0);  
}
```

Se c'è un solo output, è il colore

nome definito dall'utente

GLSL

Compilare gli shader

Gli shader vengono caricati nel programma come stringhe, compilati, passati alla GPU, ne viene fatto il linking e sono pronti per essere usati.

```
const char* vertex_shader = ". . ."; // Vertex shader code

const char* fragment_shader = ". . ."; // Fragment shader code

// Manage vertex shader
GLuint vert_s = glCreateShader (GL_VERTEX_SHADER); // Create
glShaderSource (vert_s, 1, &vertex_shader, NULL); // Attach source
glCompileShader (vert_s); // Compile

// Manage fragment shader
GLuint frag_s = glCreateShader (GL_FRAGMENT_SHADER); // Create
glShaderSource (frag_s, 1, &fragment_shader, NULL); // Attach source
glCompileShader (frag_s); // Compile

// Create a GPU program, attach to it both shaders, then link it.
GLuint shader_program = glCreateProgram();
glAttachShader (shader_program, frag_s);
glAttachShader (shader_program, vert_s);
glLinkProgram (shader_program);

// The shaders are now active inside the GPU pipeline!
```

... Finalmente si disegna!

```
while (open_window) {  
    // Clear the color buffer (default black)  
    glClear (GL_COLOR_BUFFER_BIT);  
  
    // The current vertex array is the one used for rendering  
    glBindVertexArray (vao);  
  
    // Render!  
    glDrawArrays (GL_TRIANGLES, 0, 3);  
}
```

- `glDrawArrays()`

- Manda alla pipeline i dati del **VAO** corrente, assemblandoli in primitive del tipo **GL_TRIANGLES**, partendo dal primo vertice e mandando 3 vertici in totale.
- Con `glDrawArrays()` si potrebbe disegnare anche un sottoinsieme dei vertici (partendo oltre lo zero e/o dando un numero di vertici minore del totale).

Capitolo 2 - Refactoring

Soluzione: `02_refactor.cc`

Piano di lavoro

- Il programma fin qui consiste nel solo `main()`
- Facciamo un refactoring del codice
- Creiamo tre classi:
 1. **Setup**: contiene tutte le istruzioni di inizializzazione e creazione di finestra e contesto per SFML e OpenGL
 2. **Scene**: contiene la scena codificata da **VAO** e **VBO** e le istruzioni per caricarla
 3. **Shaders**: carica e compila gli **shader**
- Creiamo una funzione `draw()` che fa il rendering
- Il `main()` ora consiste di un unico event loop che richiama la `draw()`

Capitolo 3 - Hot loading degli shader

Soluzione: `03_hotshaders.cc`

Piano di lavoro

- Gli shader sono scritti in **glsl** e vengono caricati e compilati sulla GPU a run time
- Questo permette di scrivere gli **shader** in file separati (`03_vertex.vert` e `03_fragment.frag`) e di scrivere un programma che li carica e compila “al volo” mentre il programma gira (**hot load**)
- Modifichiamo la classe **Shaders** per gestire questa funzionalità e il programma per consentire l’hot load su richiesta dell’utente
- Questo ci permetterà di sperimentare diversi shader senza dover modificare e ricompilare il programma

Modulo header-only

- Spostiamo la classe **Shaders** in un nuovo modulo header: `03_hotshaders.hh`
- Nella classe **Shaders**:
 - estraiamo dalla `load` tutte le istruzioni per la compilazione degli shader e le mettiamo in una nuova funzione `compile_attach_link()`
 - aggiungiamo un overload della funzione `load()`, con parametri espliciti, che riceve in input i nomi di due file contenenti gli shader, li carica e li compila
 - aggiungiamo un costruttore esplicito e una funzione `reload()` che usano questa versione della `load()`

Hot shaders

- Nel programma principale:
 - aggiungiamo una callback handle che associa alla pressione della barra spaziatrice il reload degli shader
 - gestiamo questa callback nel main loop
 - mantenendo gli shader minimali, possiamo ad esempio cambiare il colore con cui viene disegnato il triangolo mentre il programma gira, editando il fragment shader e attivando il reload

Capitolo 4 - Uso delle uniform

Soluzione: `04_uniforms.cc`

Piano di lavoro

- Introduciamo i concetti di `const` e `uniform` negli shader
- Modifichiamo il `fragment shader` prima con `const` e poi con `uniform` per definire il colore
- Usiamo la versione dello shader che usa `uniform` per cambiare il colore in tempo reale, alternando due colori a frequenza 1hz

Costanti negli shader

- GLSL permette di definire valori **costanti**
- Sintassi GLSL: `const int foo = 1`
- Equivalente a fare in C: `#define F00 1`
- Equivalente a fare in C++: `constexpr int foo = 1`
- I `const` GLSL sono immutabili e noti a compile time, a differenza dei `const` C++, che sono immutabili, ma risolti a run time

```
const vec3 GREEN = vec3(0.1 , 0.8 , 0.0);  
  
out vec4 frag_color;  
  
void main(void) {  
    frag_color = vec4(GREEN , 1.0);  
}
```

FRAGMENT
Shader

Uniform: passare dati verso gli shader

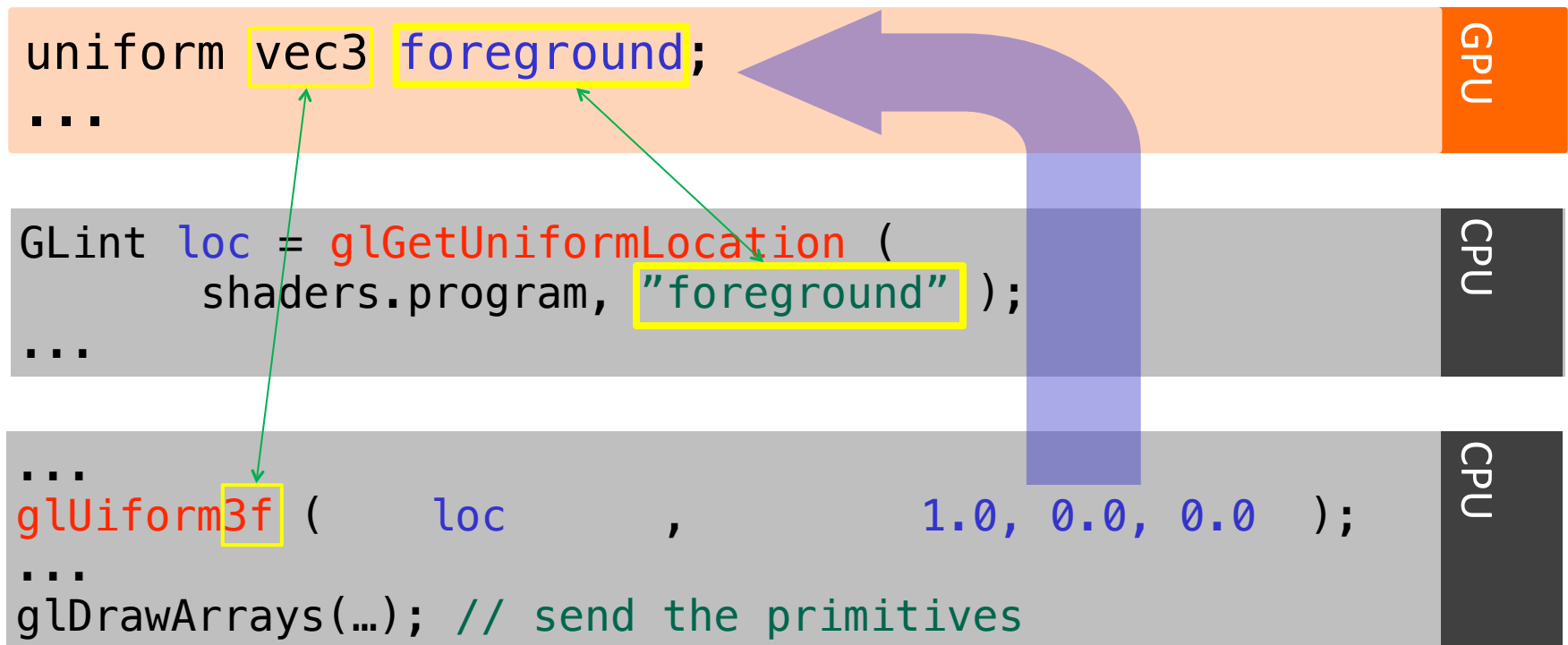
- Le **uniform** sono invece parametri dello **shader**:
 - definiti dal programma CPU, che può modificarli in qualsiasi momento
 - uguali per tutti i vertici / frammenti
 - valori decisi a run time prima del rendering

```
uniform vec3 foreground;  
  
out vec4 frag_color;  
  
void main(void) {  
    frag_color = vec4(foreground , 1.0);  
}
```

FRAGMENT
Shader

Uniform: passare dati verso gli shader

- **Binding** delle uniform “**by name**”
- (nelle versioni più recenti di OpenGL/GLSL si può fare anche esplicito, come per gli attributi)



Cambio colore a run time

- Ridefiniamo la funzione `draw()` in modo che cambi il colore della uniform a ogni secondo:
 - riceve un parametro `elapsed` che misura il tempo trascorso dall'ultimo render
 - usa un `accumulatore` del tempo trascorso per gestire le transizioni
 - quando `l'accumulatore` raggiunge il tempo desiderato, cambia il colore della uniform e si resetta
- Il `main()` usa un `clock`, di cui fa reset ad ogni ciclo, per indicare alla `draw()` il tempo trascorso in un ciclo

Capitolo 5 - Colori sui vertici

Soluzione: `05_multicolor.cc`

Piano di lavoro

- Modifichiamo la classe **Scene** aggiungendo il colore come attributo di ogni vertice
- Modifichiamo il **vertex shader** per ricevere il colore come ulteriore input e passarlo come attributo da interpolare al rasterizzatore
- Modifichiamo il **fragment shader** per ricevere in input il colore interpolato
- Usiamo una **uniform** nel **fragment shader** per modulare il colore interpolato

Come aggiungere attributi ai vertici

- Se per ogni vertice abbiamo:
 - X, Y, Z (geometria = attributo con indice 0)
 - R, G, B (colore = attributo con indice 1)
- Possiamo organizzare i dati in vari modi:
 - buffer interlacciato (interleaved):
 $X_0, Y_0, Z_0, R_0, G_0, B_0, X_1, Y_1, Z_1, R_1, G_1, B_1, \dots$
 - buffer concatenati:
 $X_0, Y_0, Z_0, X_1, Y_1, Z_1, \dots R_0, G_0, B_0, R_1, G_1, B_1,$
 - o anche:
 $X_0, X_1, \dots Y_0, Y_1, \dots Z_0, Z_1, \dots R_0, R_1, \dots G_0, G_1, \dots$
 - Potrebbero anche esserci due VBO, uno per le posizioni, l'altro per i colori.
 - Et cetera...



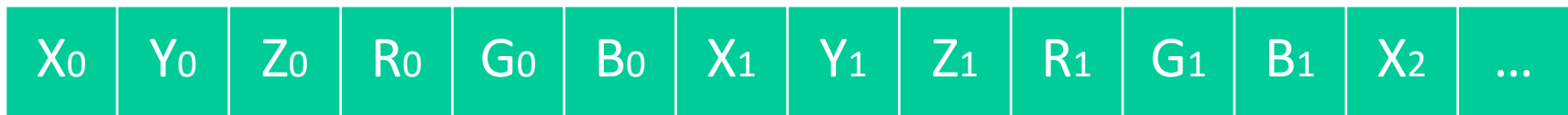
scegliamo
questo

Aggiungiamo l'attributo colore

- Definiamo i dati. Aggiungiamo il colore di ogni vertice nel VBO, facendo seguire una tripletta RGB a ogni tripletta XYZ:

```
points = {  
    0.0, 0.5, 0.0, // x,y,z of first point.  
    1.0, 0.0, 0.0, // rgb  
    0.5, -0.5, 0.0, // x,y,z of second point.  
    0.0, 1.0, 0.0, // rgb  
    -0.5, -0.5, 0.0, // x,y,z of third point.  
    0.0, 0.0, 1.0, // rgb  
};
```

- Struttura risultante:



- Ora bisogna informare il **VAO** dell'esistenza di due attributi, interleaved nello stesso **VBO**.
- Due chiamate a **glVertexAttribPointer**, la prima per l'attributo 0, la seconda per l'attributo 1.

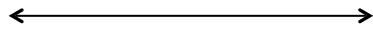
glVertexAttribPointer

- Il parametro **stride** indica il numero di byte di cui bisogna spostarsi per passare all'elemento successivo (per noi, per muoversi di vertice in vertice).
- Il parametro **offset** deve indicare il numero di byte a cui viene trovato l'attributo dentro l'elemento (per noi, la tripletta di posizioni XYZ oppure RGB del vertice).
- **Attenzione:** per ragioni oscure, il parametro offset invece di essere un tipo size come stride, è dichiarato come un puntatore a void e per questo richiede un casting. Ciononostante, internamente OpenGL lo utilizza come un offset.

```
glVertexAttribPointer (GLuint index,           // attribute location.  
                      GLint size,             // number of components.  
                      GLenum type,            // type of components.  
                      GLboolean normalized,   // if integer, normalize or not?  
                      GLsizei stride,         // stride.  
                      const void * pointer);  // offset. warning: not a pointer
```

Aggiungiamo l'attributo 0: posizione

n. elem. = 3 (float)



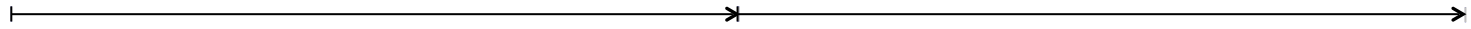
VBO:



stride:

6 x 4 bytes

6 x 4 bytes



offset:

0 bytes



Attiviamo gli attributi

- Attiviamo gli attributi nel **VAO**:

```
glEnableVertexAttribArray (0);    // use location 0 for positions
glEnableVertexAttribArray (1);    // use location 1 for colors
// attribute locations must be matched in the vertex shader!
```

- Quando il rendering viene lanciato con questo **VAO** attivo, il **vertex puller** saprà che deve trasferire questi attributi alla pipeline

Modifichiamo la uniform per modulare il colore

- Modifichiamo la uniform nella classe `Scene`. La chiamiamo `mod_color_location`
- Aggiungiamo due campi costanti `darken_color` e `lighten_color` per i colori di modulazione
- useremo la uniform `mod_color` nel `fragment shader` per combinarla col colore interpolato dai vertici
- Modifichiamo la chiamata a `glGetUniformLocation` nel `main`, per ottenere la location della nuova uniform nello shader.
- nella funzione `draw` alterniamo il valore della uniform con il passare dei secondi.

Passaggio dati dal vertex al fragment shader

- Binding by name per il passaggio dati fra gli shader:
 - stesso attributo, stesso nome in entrambi gli shader
 - **out myname** per il vertex shader, **in myname** per il fragment shader
 - è possibile raggruppare gli attributi in “interface blocks” (simili alle struct del C)

Vertex Shader

- input: attributi da VAO/VBO, presi dal vertex puller.
- output: attributi custom per il rasterizzatore, da **interpolare**.
- obbligatorio **gl_Position** (vec4)
- (**gl_Position** è un nome riservato da OpenGL, obbligatorio per la pipeline)

Fragment Shader

- input: attributi custom dal rasterizzatore, **interpolati**.
- output: attributi per l’output combiner.
- Ad esempio: **frag_color** (vec4)
- (Se il fragment shader ha una sola variabile out, OpenGL la passa al color buffer qualsiasi nome abbia)

Negli shader:

VERTEX
Shader

```
layout(position = 0) in vec3 position;  
layout(position = 1) in vec3 vertex_color;
```

```
out vec3 interpolated_color;
```

```
void main(void) {  
    gl_Position = vec4( position, 1.0 );  
    interpolated_color = vertex_color ;  
}
```

L
I
N
K

FRAGMENT
Shader

```
in vec3 interpolated_color;
```

```
uniform vec3 mod_color;  
out vec4 fragment_color;
```

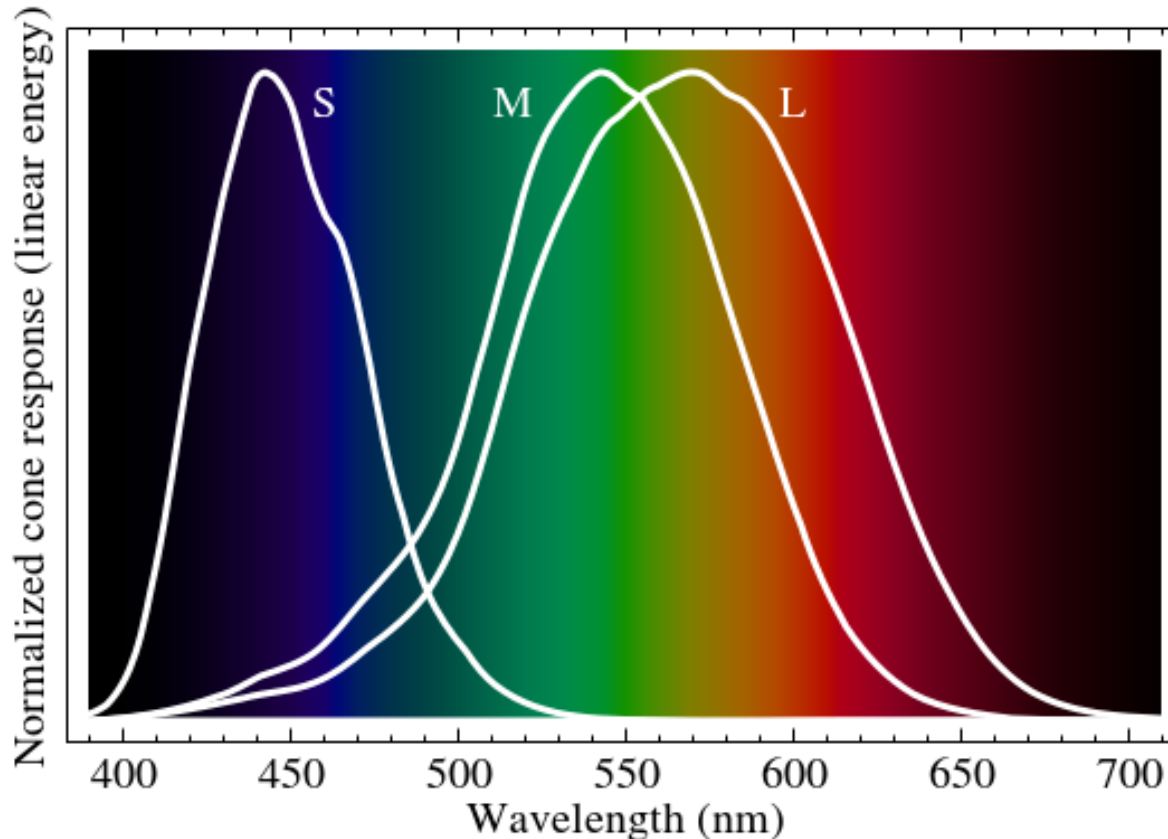
```
void main(void) {  
    fragment_color = vec4(interpolated_color +  
                           mod_color, 1.0);  
}
```

Note sul colore

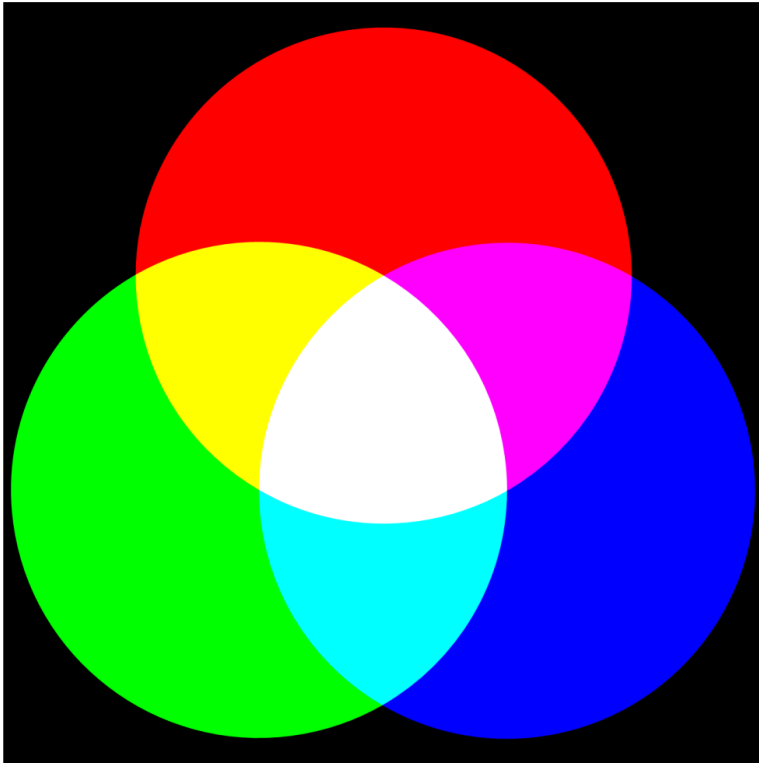
- Abbiamo usato offset costanti (± 0.2 su tutte le componenti) per scurire o schiarire il colore
- In realtà, sommare colori in spazio RGB non ha molto senso:
 - Si potrebbe eccedere l'intervallo consentito $[0,1]$ per le componenti (ma a questo si può rimediare facendo clamp all'intervallo consentito)
 - Il vero problema è che il risultato non è un colore prevedibile, e non è affatto ovvio come operare sul colore per ottenere effetti che siano percepiti come simili. La stessa operazione, che sembra funzionare su un colore, potrebbe produrre risultati completamente diversi o addirittura opposti su un altro.
 - Ad esempio: scurire/schiarire; saturare/desaturare; o peggio ancora correzioni di colore per cambiare temperatura o tinta.

Note sul colore: percezione umana

- Luce: spettro visibile della radiazione elettromagnetica
- La retina “campiona” tre picchi (short: blu, medium: green, long: red) e deriva il resto
- Per noi, una sorgente 570nm (gialla) è indistinguibile da due sorgenti 530nm (verde) e 650nm (rossa), vicine fra di loro, e di pari intensità combinata: la luce la vediamo gialla in entrambi i casi



Note sul colore: colori primari e complementari

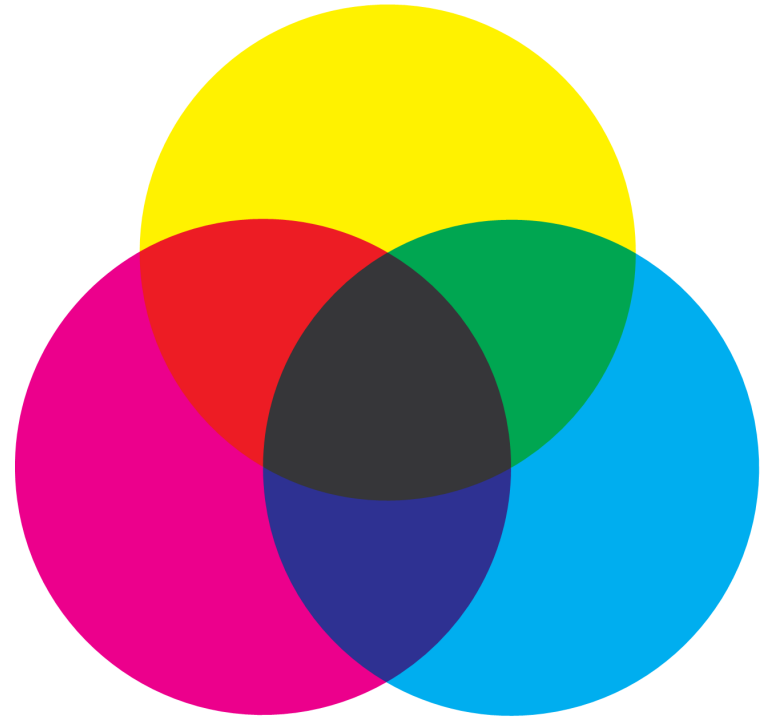


Modello additivo

Schermo spento: nero

Ogni pixel ha 3 luci RGB (colori primari), le tre sorgenti luminose possono essere accese ognuna con una sua intensità.

Accese tutte e tre: bianco



Modello sottrattivo

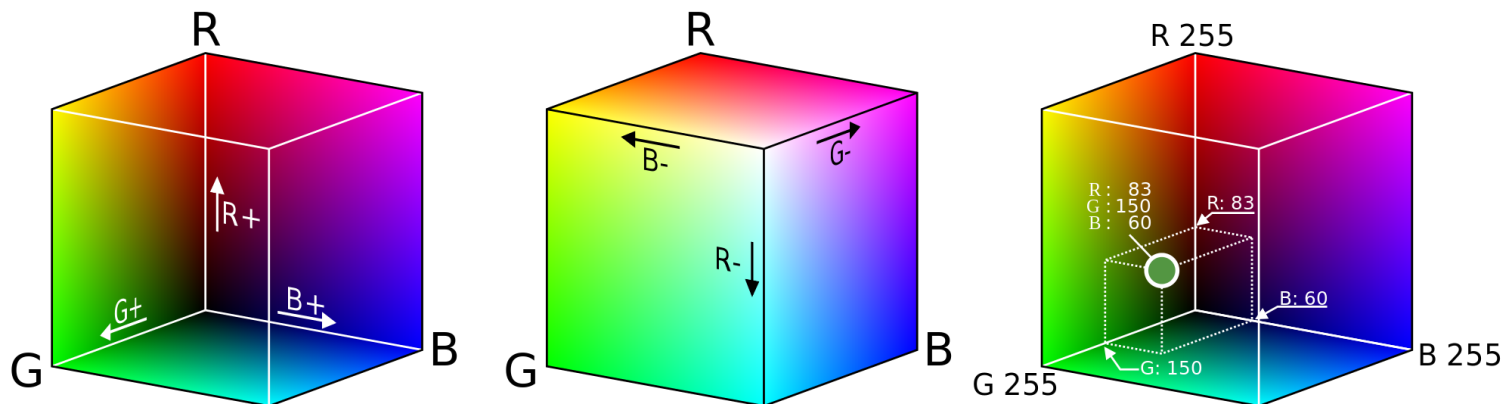
Foglio vuoto: bianco.

Tre pigmenti CYM (colori complementari), ognuno di loro **NON** riflette un colore primario RGB, intensità tramite densità.

Pennellati tutti insieme: nero

Note sul colore: RGB

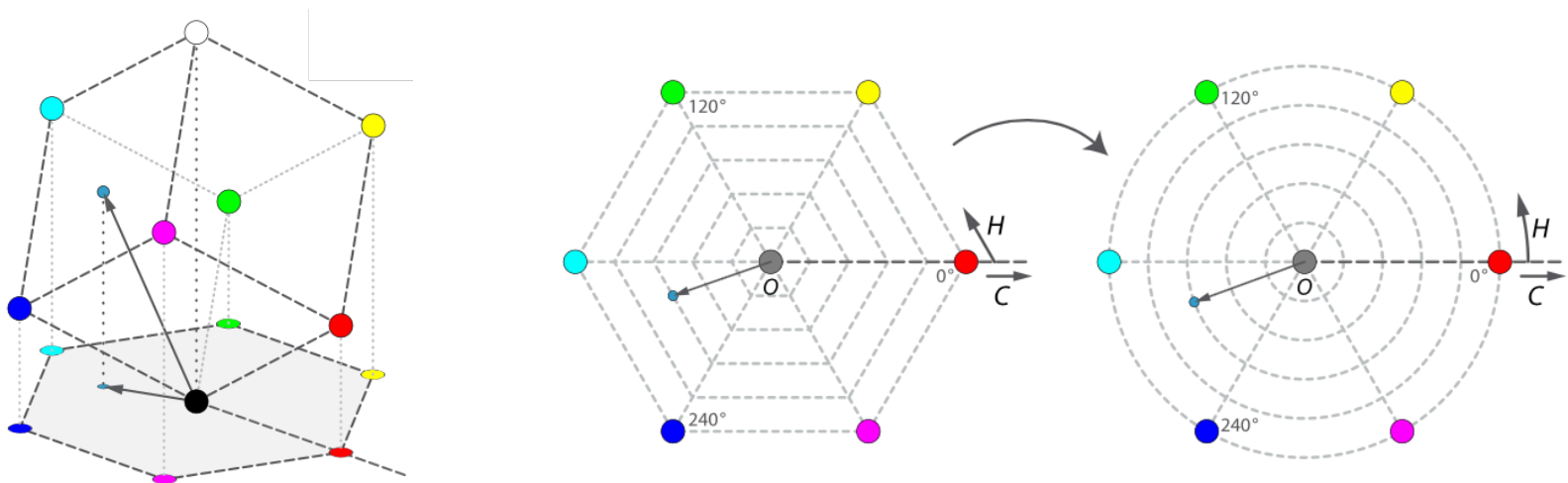
- Lo spazio RGB si rappresenta tramite un cubo di lato 1:
 - nero nell'origine e bianco al vertice opposto
 - i colori primari rosso, verde e blu sui vertici connessi al nero
 - i colori complementari giallo, magenta e ciano sui vertici connessi al bianco
 - ogni punto interno al cubo corrisponde al colore dato dalle sue coordinate
 - non è intuitivo capire che colore viene fuori da una certa miscela RGB



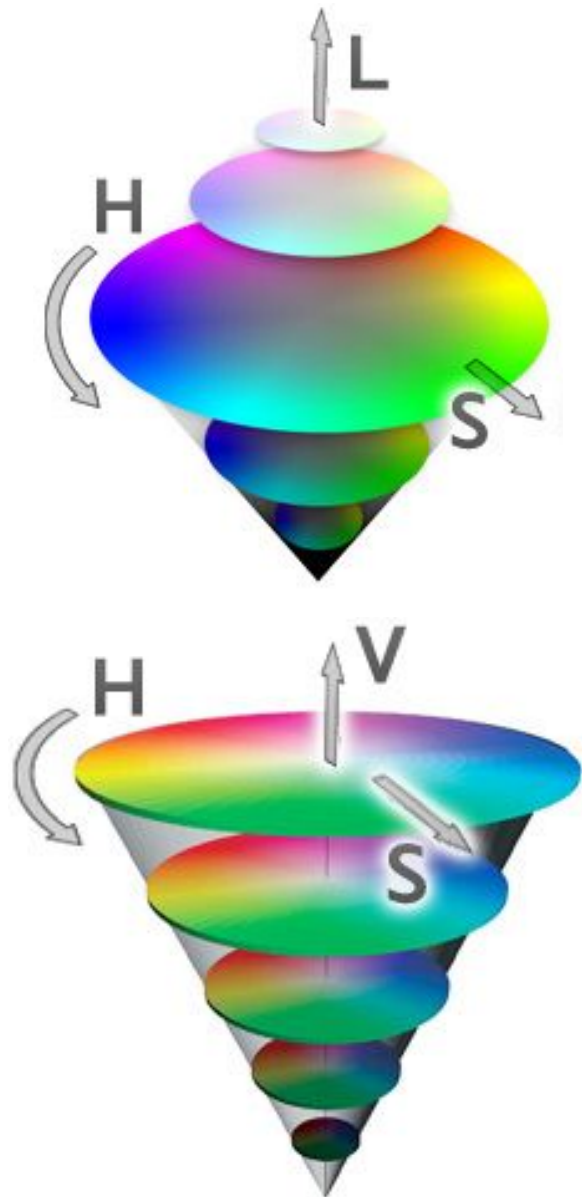
Note sul colore: HSL e HSV

- Esistono diversi altri spazi colore
- Due molto usati sono gli spazi HSL e HSV
- Si ottengono proiettando lo spazio RGB su un cono o un doppio cono
- Esistono semplici formule di conversione tra spazi diversi

<https://progosling.com/en/color-converters>



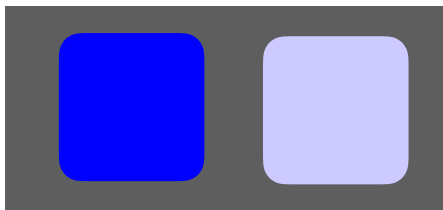
Note sul colore: HSL e HSV



- **H: Hue**, tinta del colore (coordinata angolare)
- **S: Saturation**, intensità del colore (lineare in $[0,1]$)
- **L in HSL: Lightness** luminosità, quanta “luce” viene aggiunta o tolta al colore, da 0 (tutto nero) a 1 (tutto bianco). Colori “puri” con $L=0.5$, $S=1$
- **V in HSV: Value** (or **brightness**), brillantezza, quanta “forza” ha il colore da 0 (tutto nero) a 1 (più colorato possibile). Colori “puri” con $V=1$, $S=1$
- Questi spazi permettono, rispetto a RGB, di fare alcune operazioni base. Si può provare a interpolare fra i colori (senza troppe aspettative).
- Per scurire/schiarire dovrebbe bastare diminuire/aumentare la componente **L** o **V**
- Per saturare/desaturare, la componente **S**.

Difetti di HSL e HSV

- Sono spazi veloci e semplici da capire e computare, ma hanno molti difetti di imprevedibilità e risultati sballati, sono utilizzabili accettandone le limitazioni
- Per fare meglio, bisogna andare verso spazi colore molto più complicati, come CIE $L^*a^*b^*$
- **PROBLEMI!**
 - Assenza di linearità percettiva. Cambiare **H** da solo causa cambiamenti percepiti di luminosità e saturazione, anche se **L/V** e **S** sono inalterate
 - In entrambi gli spazi **S+L** e **S+V** rimangono dimensioni legate fra loro con risultati molto poco intuitivi, non sono realmente disaccoppiate. **S** si chiama saturazione in entrambi gli spazi ma fa cose molto diverse
 - Risultati paradossali! Prendiamo un blu puro, trasformato cambiando **S** in HSV e **L** in HSL, per entrambi i colori (blu e trasformazione) vale che:



#0000FF #CDCAFF

- $H=243$ (ma la tonalità percepita è cambiata)
- In HSV: $V=1$, ma la luminosità percepita è diversa
- In HSL: $S=1$, ma la saturazione percepita è diversa

Difetti di HSL e HSV

sRGB



CIE: L^*



HSL: L



HSV: V



Capitolo 6 - Struttura indicizzata

Soluzione: `06_indexed.cc`

Piano di lavoro

- Aggiungiamo alla classe **Scene** un nuovo dato: un vettore di interi, indici validi nel vettore di vertici.
- Tre indici fanno una faccia.
- Aggiungiamo una nuova componente in memoria video: L'**Element Buffer Object (EBO)**, che conterrà gli indici. Va aggiunto tramite **binding** al **VAO**
- Poi si modifica la **draw()** per disegnare i triangoli usando gli indici

Element Buffer Object (EBO)

Nuovi campi dentro la classe `Scene`

```
std::vector<unsigned int> indices;  
GLuint vbo;
```

Inizializzazione e binding dentro `Scene::Load()`

```
// 1 face, 3 indices per face, CCW order  
indices = {  
    0, 2, 1 // CCW (counterclockwise)  
};  
// . . .  
glGenBuffers(1, &ebo);  
// MUST be bound after the VAO's binding!  
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo);  
// copy to video RAM  
glBufferData(GL_ELEMENT_ARRAY_BUFFER,  
             indices.size() * sizeof(unsigned int),  
             indices.data(),  
             GL_STATIC_DRAW);
```

Disegno dentro `draw()`

```
glDrawElements(GL_TRIANGLES, scene.indices.size(), GL_UNSIGNED_INT, 0);
```

Element Buffer Object (EBO)

- Il registrazione dell'EBO dentro al VAO segue regole diverse dal VBO
- Un VBO viene registrato nel VAO quando viene chiamata `glVertexAttribPointer()`, contestualmente alla descrizione del suo attributo interno di cui il VAO memorizza il layout.
- Vengono utilizzati il VBO e il VAO al momento correnti (bound) nello stato di OpenGL
- Un EBO, invece, viene registrato nel VAO corrente ogni qual volta ne viene fatto binding nello stato OpenGL
- Un VAO può avere più attributi su VBO diversi, ma in un dato momento può essere registrato ad uno e un solo EBO

Capitoli 7 e 8: Culling e Depth test

Soluzioni: `07_culling.cc`
`08_zbuffer.cc`

Piano di lavoro

- Culling: aggiungere due nuovi vertici e due nuove facce alla nostra struttura (classe **Scene**)
- Culling: abilitare dentro il **main()** la funzionalità di culling, sperimentare cosa succede cambiando l'ordine dei vertici nelle facce
- Zbuffer: aggiungere un nuovo triangolo (3 vertici e loro indici), parzialmente sovrapposto alla struttura precedente, con Z negativa. Il triangolo è l'ultimo nel vettore degli indici (viene disegnato per ultimo)
- Zbuffer: Osservare che con il depth test abilitato, viene disegnato dietro alla struttura precedente, altrimenti davanti
- Zbuffer: tutto gestito con default ragionevoli dentro la pipeline, basta aggiungere la profondità ai vertici e abilitare il test

Considerazioni sul culling

- Il Culling è un'ottimizzazione
- Se so di stare disegnando una superficie chiusa, le facce interne alla superficie non verranno mai viste.
- Se disegno la superficie in modo che le back faces siano tutte rivolte all'interno, posso ottimizzarle via.
- Conviene eliminarle dalla pipeline il prima possibile, altrimenti dovrebbero venire rasterizzate, arrivare in fondo alla pipeline con tutti i loro frammenti... solo per venire sempre eliminate dal depth test
- Se sto disegnando una superficie aperta, con necessità di vederla da entrambi i lati, il culling non va abilitato!